

자연어처리(NLP)의 진화와 핵심 원리

기계는 어떻게 인간의 언어를 이해하게 되었는가?



자연어처리(NLP)의 두 방향

자연어 이해 (NLU)

- 검색 엔진 질의어 해석
- 감성 분석(Sentiment Analysis)



자연어 생성 (NLG)

- 챗봇 대화
- 기계번역 및 문서 요약

자연어처리 기술의 3가지 패러다임

규칙 기반 (Rule-based)

1950s~80s

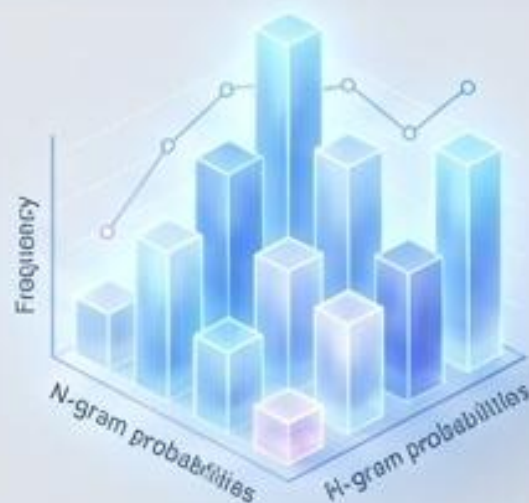


핵심: 전문가가 입력한 문법 규칙.

장단점: 특정 도메인 정확도 높음, 확장성 부족.

통계 기반 (Statistical)

1990s~2000s

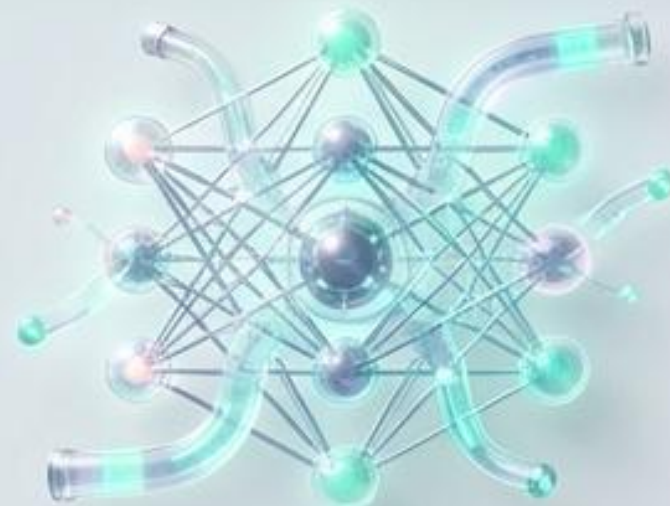


핵심: 대규모 말뭉치의 출현 빈도와 확률.

장단점: 수동 규칙 불필요, 깊은 문맥 이해 한계.

신경망 기반 (Neural-based)

2010s~



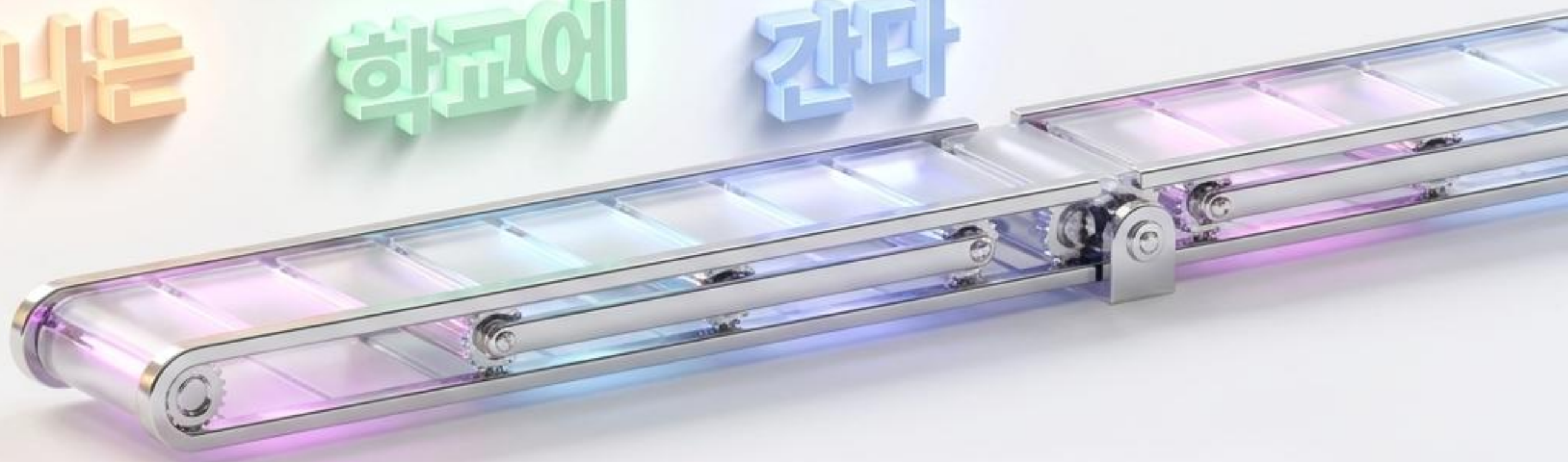
핵심: 딥러닝(RNN, Transformer)을 통한 의미 학습.

장단점: 뛰어난 성능, 대규모 데이터와 연산력 필요.

언어의 본질: 순차 데이터 (Sequential Data)

일반적인 인공지능은 입력을 한꺼번에 처리합니다. 하지만 언어는 순서가 섞이면 의미를 잃습니다.

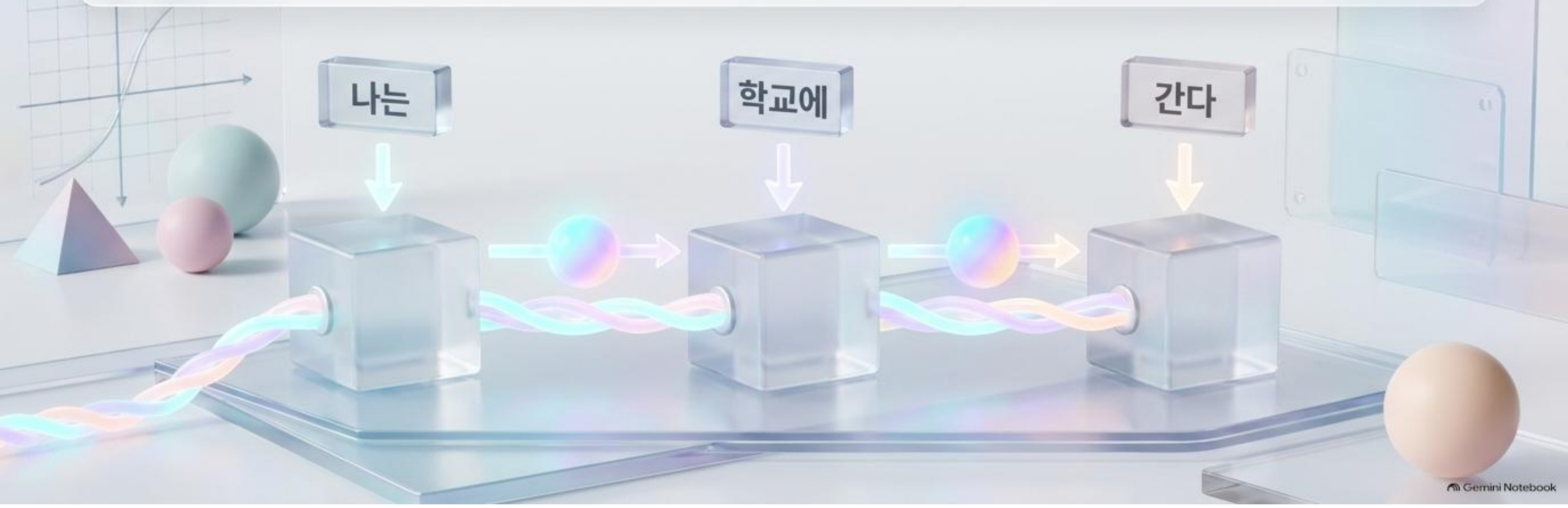
나는 학교에 간다



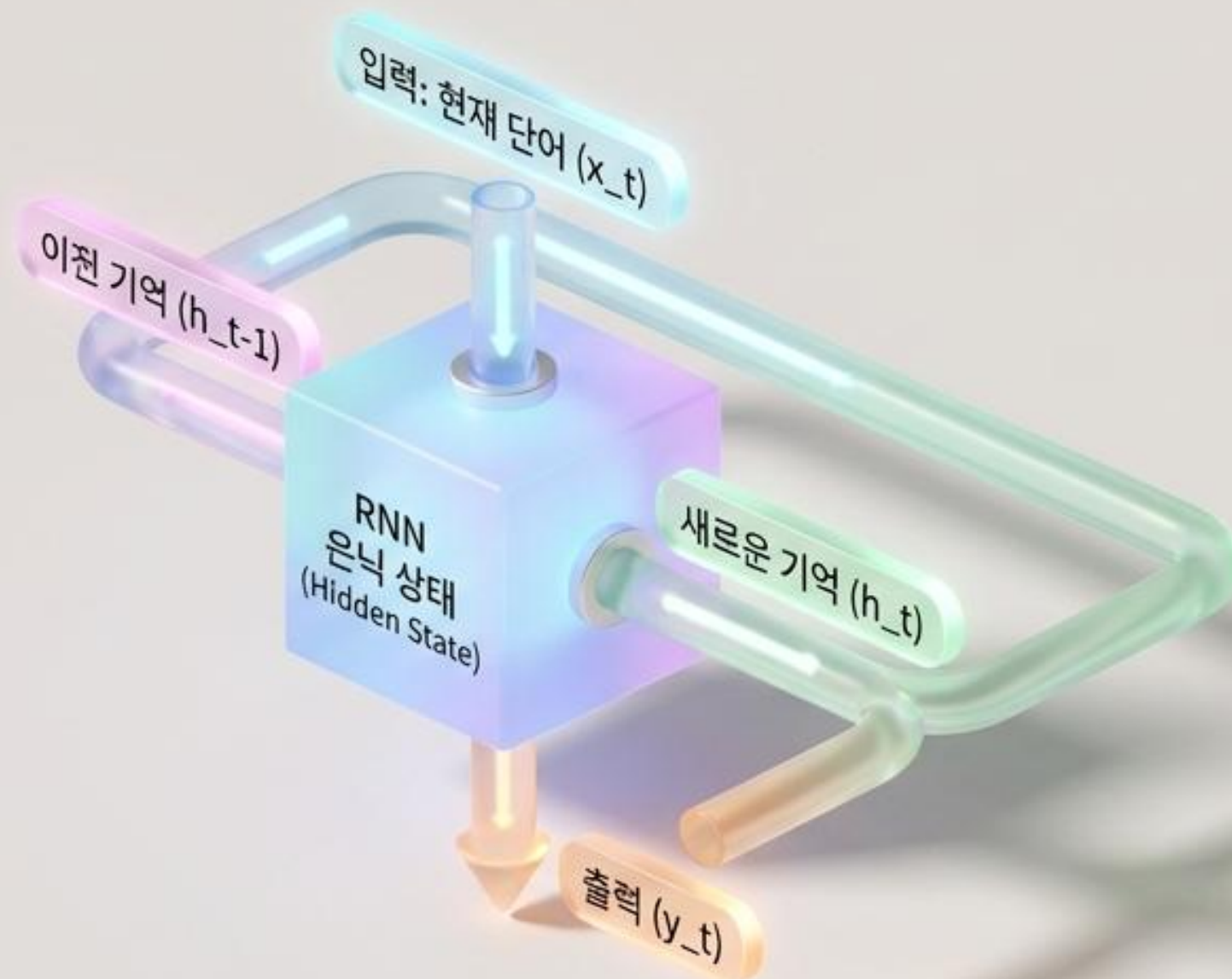
결론: 기계에게는 시간의 흐름을 '기억'할 수 있는 새로운 구조가 필요했습니다.

시간의 흐름을 기억하는 인공지능, 순환신경망(RNN)

- 언어는 단어의 '순서'가 핵심인 순차 데이터(Sequential Data)입니다.
- RNN은 이전 시점의 정보를 '은닉 상태(Hidden State)'로 저장하여 다음 단어 계산에 넘겨줍니다.
- 과거의 기억과 현재의 입력을 결합하여 문맥을 파악합니다.

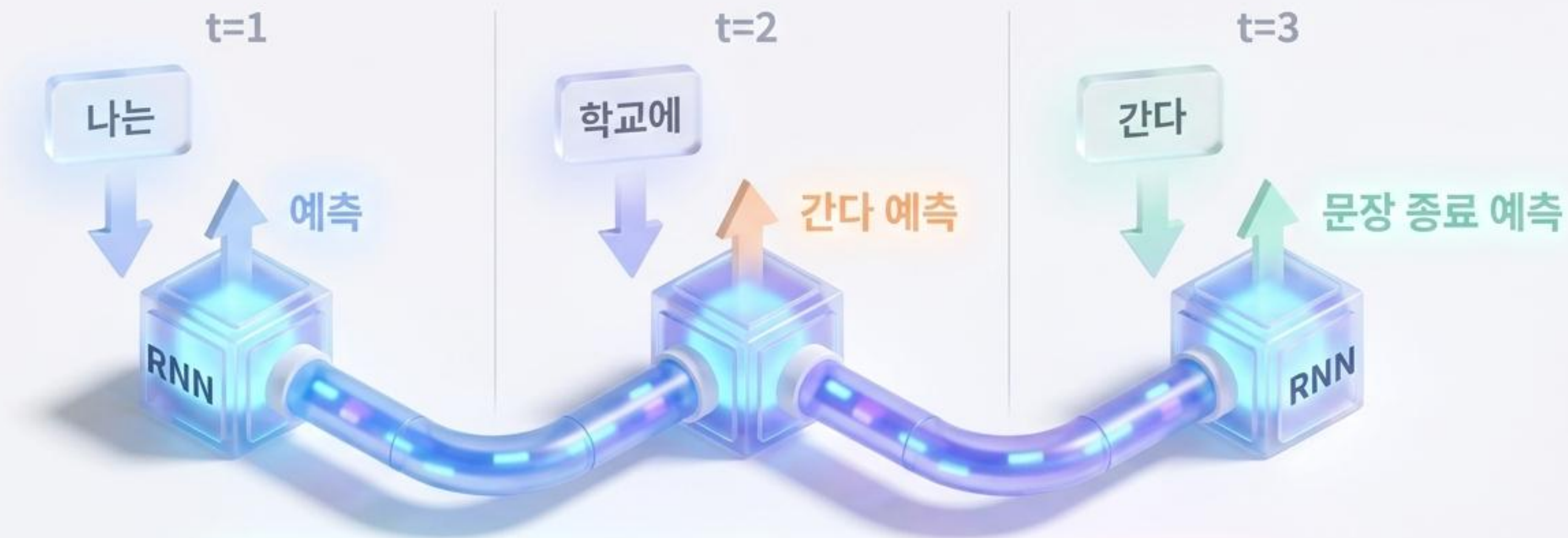


순환신경망(RNN): 기억을 품은 네트워크



핵심 원리: 매 시점(time step)마다 이전 정보를 누적하여 다음 계산에 활용합니다.

문장이 처리되는 과정 (Step-by-Step)



은닉 상태(h_t)는 이전까지 본 단어들의 정보를 압축하여 저장하는 역할을 합니다.

치명적 한계 : 장기 의존성 문제 (Long-Term Dependency)

철수는 아침에 우산을 들고 집을 나섰다.

...[시간의 흐름]...

...오후에 비가 내렸다.



문제점: 앞부분의 중요한 정보가 문장 끝에 도달할 즈음에는 희미해져 사실상 사라집니다.

숫자로 보는 기억 상실: 기울기 소실(Vanishing Gradient)

학습 과정에서 오차 신호(예: 0.9)가 반복해서 곱해질 때 발생하는 현상입니다.



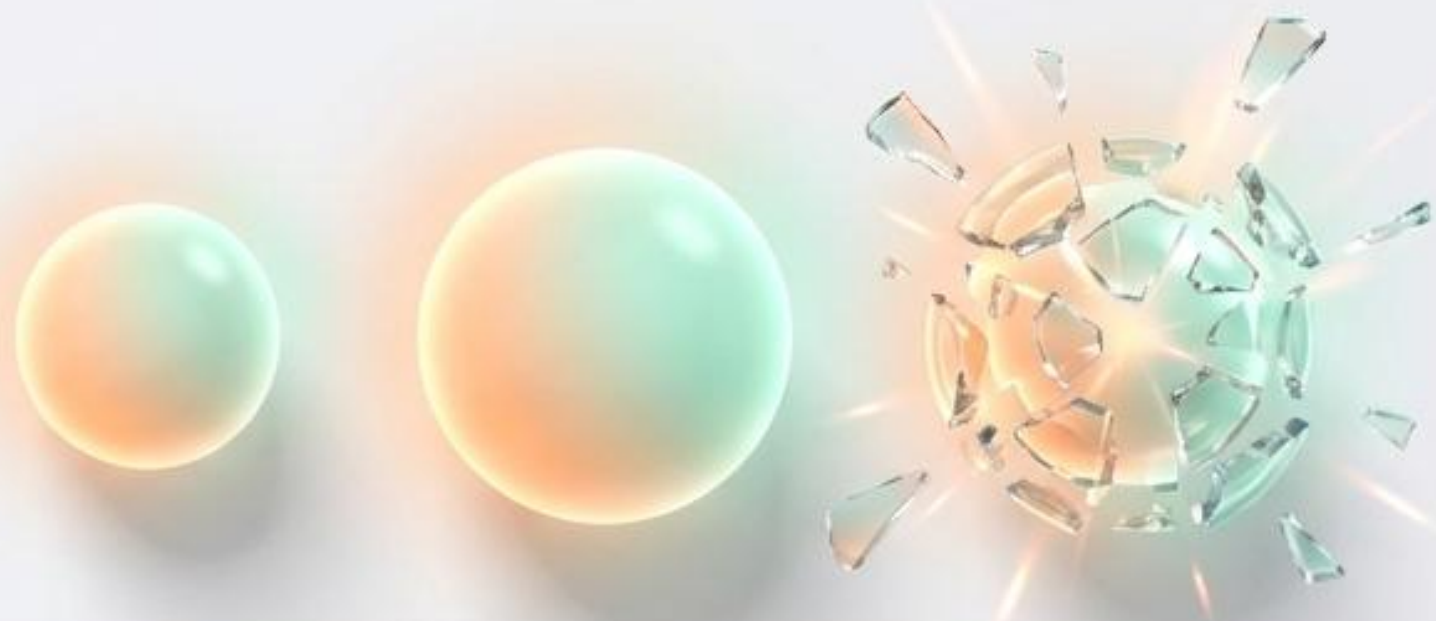
붕괴하거나, 폭발하거나

기울기 소실 (Vanishing Gradient)



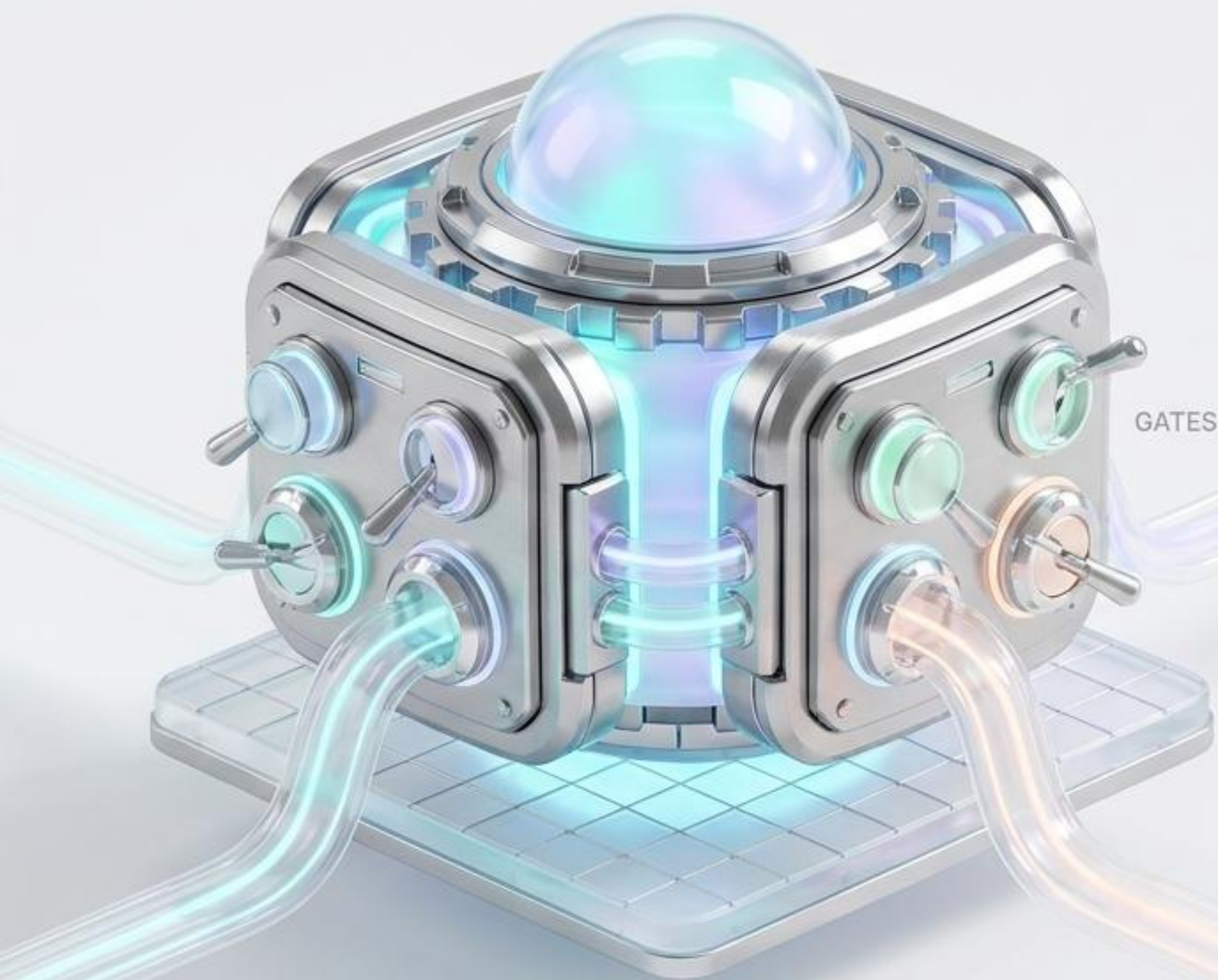
신호가 1보다 작을 때: 정보가 소멸됨.

기울기 폭발 (Exploding Gradient)



신호가 1보다 클 때: 기하급수적으로 커져 연산 붕괴.

결론: 기존 RNN 구조로는 긴 문장을 안정적으로 학습할 수 없었습니다.



기억을 금고에 보관하는 대신, LSTM과 GRU

- 장기 의존성 문제를 해결하기 위해 고안된 진화된 신경망 구조입니다.
- 정보의 보존과 삭제를 통제하는 '게이트(Gate)'와 별도의 장기 기억 장치인 '셀 상태(Cell State)'를 도입했습니다.

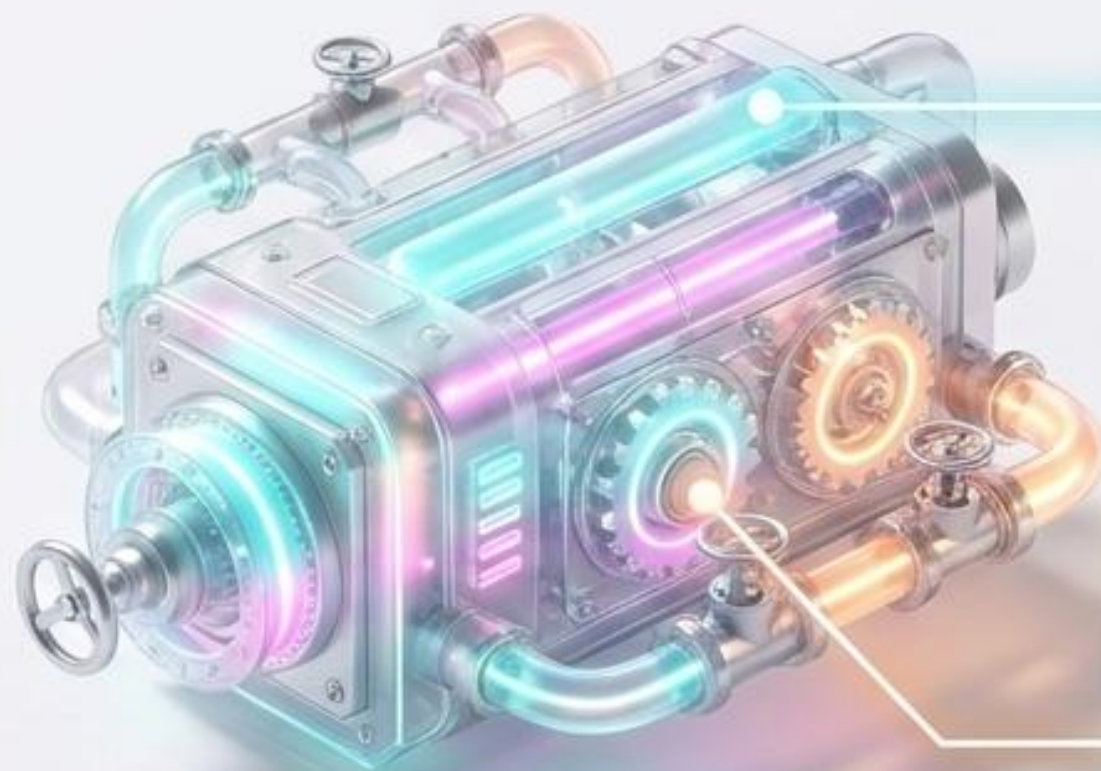
한계점: 여전히 단어를 순서대로 처리해야 하므로 병렬연산이 불가능하다는 구조적 한계는 남아있습니다.

공학적 해결책: LSTM & GRU

기존 RNN



LSTM (Long Short-Term Memory)



1



셀 상태 (Cell State):
장기 기억을 전담하는
별도의 컨베이어 벨트.

2



게이트 (Gate): 무엇을
기억하고 무엇을 잊을지
통제하는 밸브 시스템.

⚠ 한계점: 하지만 여전히 단어를 하나씩 순서대로 처리해야 하므로,
아주 긴 텍스트에서는 구조적 한계가 존재합니다.

컴퓨터가 문장을 씹어 삼키는 방법, 토큰화(Tokenization)

컴퓨터가 계산할 수 있도록 문장을 단어, 형태소, 부분단어(Subword) 단위로 쪼개는 과정.



한국어의 특징:

교착어인 한국어는 단순 띄어쓰기가 아닌 형태소 단위 분리가 필수적입니다.

주요 도구:

mecab-ko (한국어 형태소 분석),
SentencePiece (구글), tiktoken (오픈AI).

컴퓨터는 단어를 어떻게 읽는가?



기계는 텍스트를 있는 그대로 소화할 수 없습니다.
인간의 언어를 기계의 언어(수학)로 번역하는 두 가지 필수 관문이 있습니다.

1

Step 1. 토큰화 (Tokenization): 쪼개기

2

Step 2. 임베딩 (Embedding): 숫자로 바꾸기

Step 1. 토큰화 (Tokenization): 언어의 원자 단위

문장을 컴퓨터가 처리할 수 있는 가장 작은 계산 단위(토큰)로 분리하는 과정.



한국어 토큰화의 딜레마: 교착어의 특징

I am playing

단순 띄어쓰기로
완벽히 분리 가능

[I]

[am]

[playing]

강아지가
강아지를
강아지와

조사와 어미가 결합하는 '교착어'
- 띄어쓰기만으로는 분리 불가능

강아지

가

를

와

해결책: 부분단어(Subword) 분리 기술 적용.

한국어 특화(mecab-ko), 구글(SentencePiece), 오픈AI(tiktoken) 등 활용.



Step 2. 워드 임베딩 (Word Embedding): 단어를 숫자로

대규모 말뭉치(Corpus)

→ 어휘 집합(Vocabulary) 생성

→ 고유한 벡터(Vector) 할당



임베딩이란? 단어의 단순한 인덱싱을 넘어, 단어의 의미를 수십~수백 개의 실수(Real Numbers)로 이루어진 다차원 배열로 매핑하는 기술입니다.

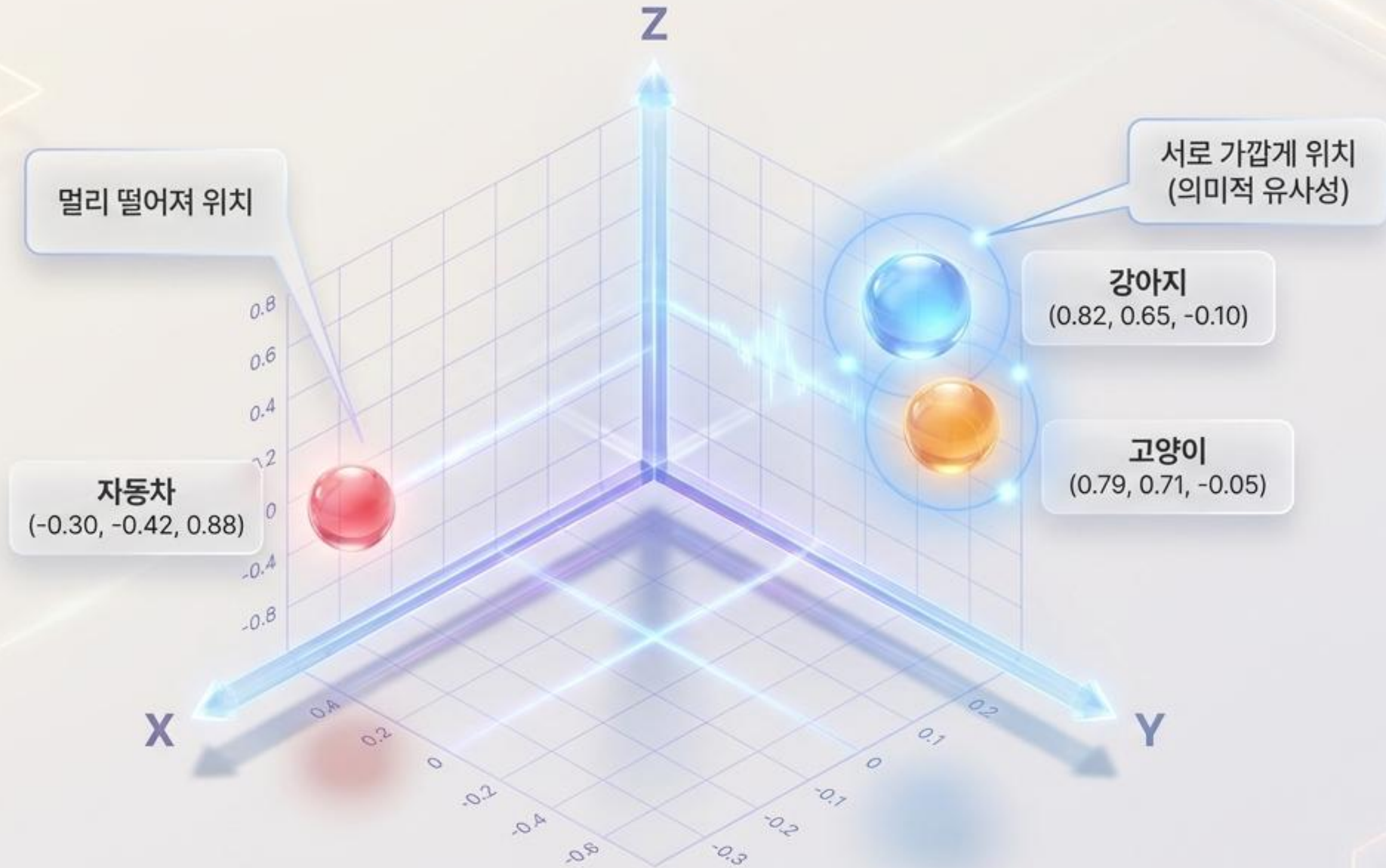
문맥이 곧 의미다: Word2Vec (2013)

구글이 발표한 혁신적 방법. 주변에 자주 등장하는 단어를 예측하도록 학습합니다.



비슷한 문맥에서 쓰이는 단어는 100~300 차원의 벡터 공간에서 가까운 위치에 놓이게 됩니다.

3차원 벡터 공간의 시각화

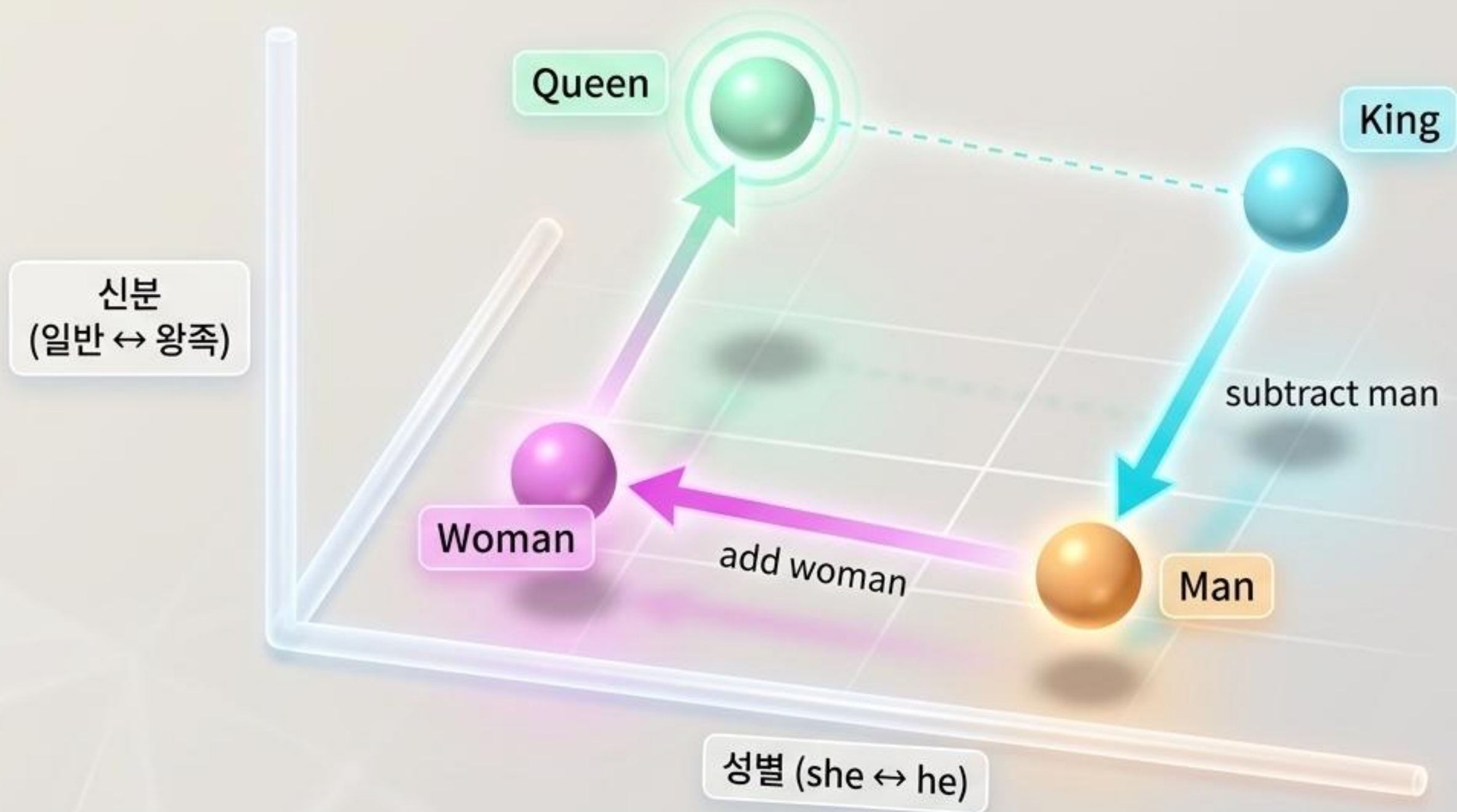


벡터 연산의 마법: 숫자에 담긴 논리

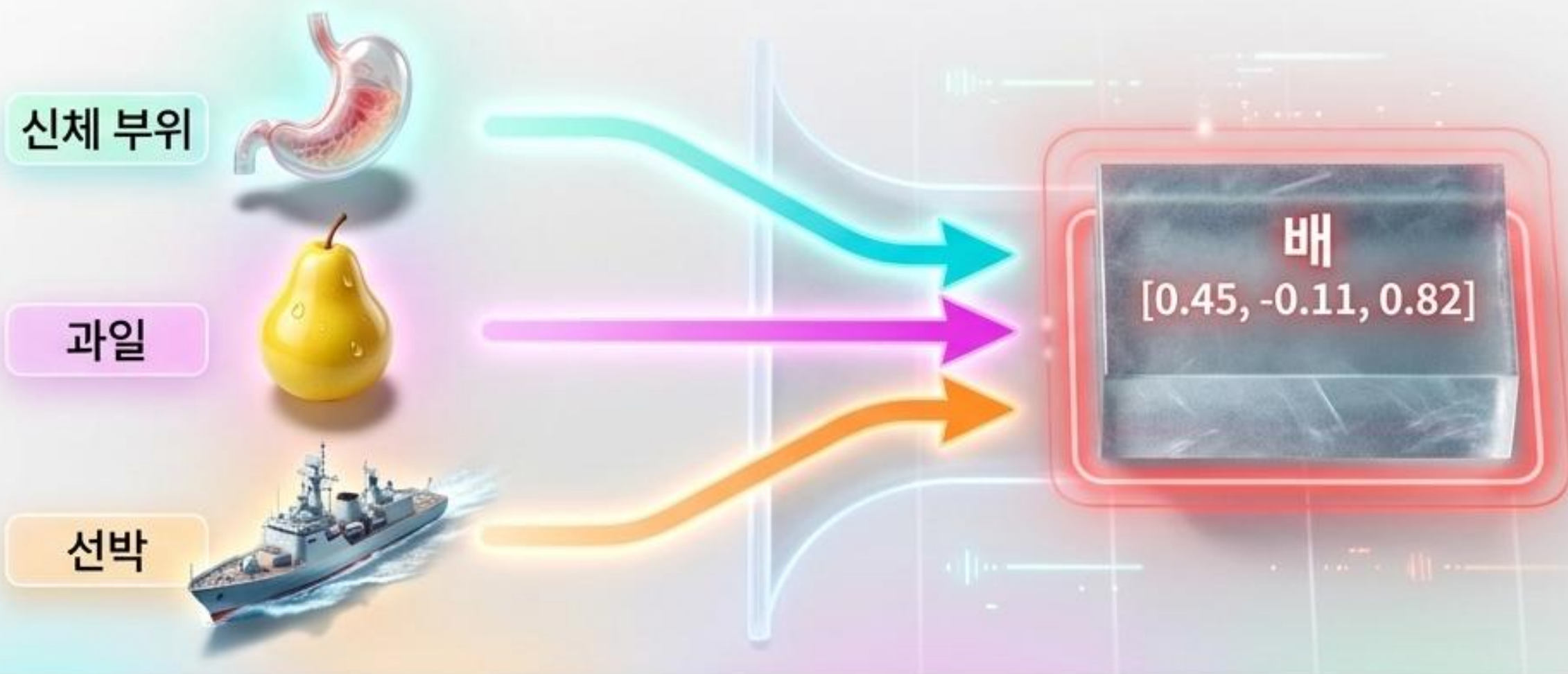


단어를 단순히 숫자로 바꾼 것이 아니라, 임베딩 벡터의 덧셈과 뺄셈을 통해 단어 사이의 의미적, 문법적 관계가 기하학적 구조 속에 완벽히 담겨 있음을 증명한 결정적 사례입니다.

공간으로 증명하는 개념의 축 (Word2Viz)



끝나지 않은 한계: 정적 임베딩 (Static Embeddings)



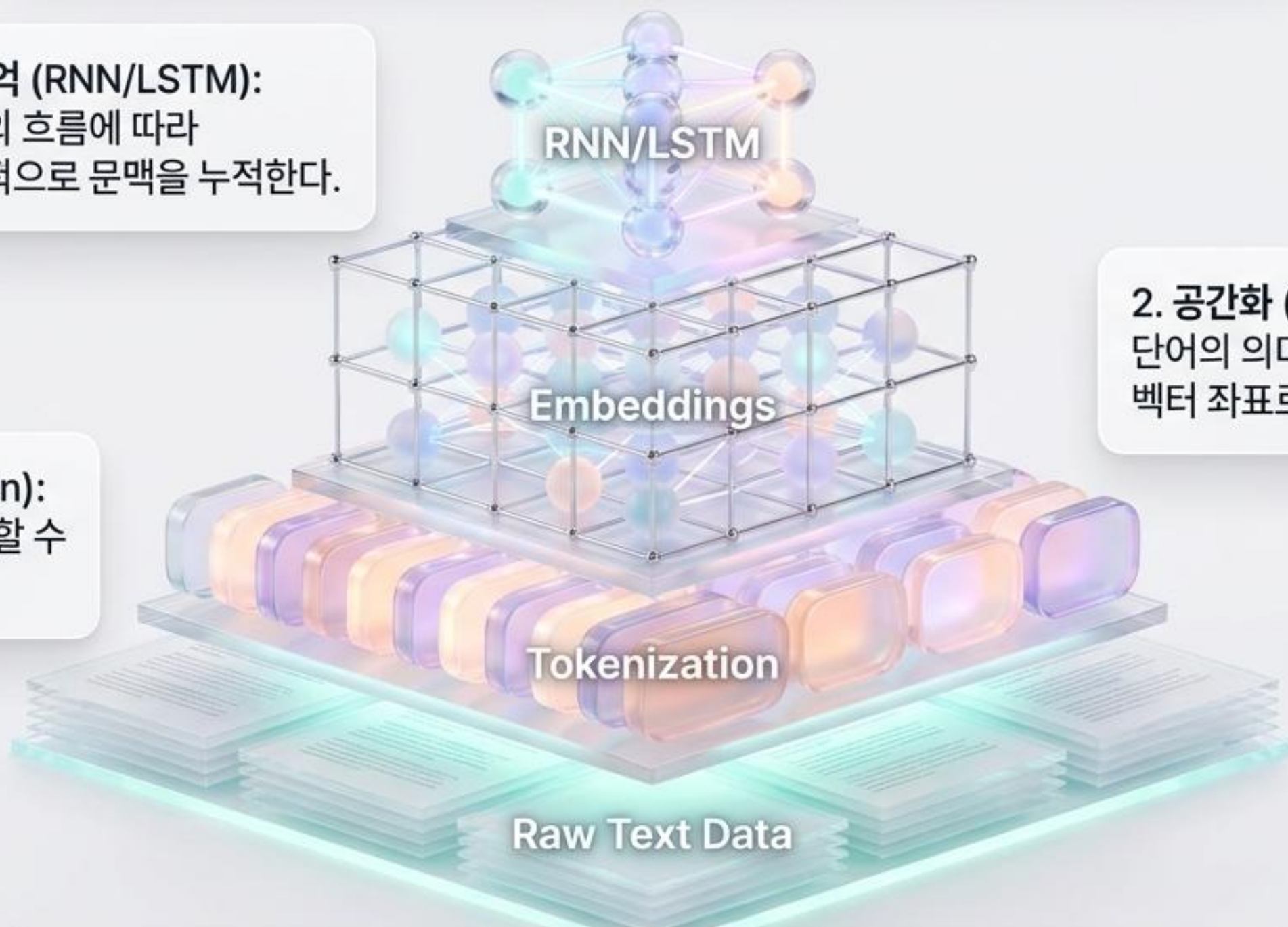
동음이의어의 딜레마: 문맥이 전혀 달라도 항상 똑같은 벡터값을 가집니다.
기계는 아직 중의성(Ambiguity)을 파악하지 못합니다.

거대언어모델(LLM)을 탄생시킨 기반 기술의 결합

3. 기억 (RNN/LSTM):
시간의 흐름에 따라
순차적으로 문맥을 누적한다.

2. 공간화 (Embedding):
단어의 의미와 관계를
벡터 좌표로 변환한다.

1. 분해 (Tokenization):
문장을 컴퓨터가 소화할 수
있는 단위로 쪼갬다.



이 세 가지 기반 기술의 발전이 오늘날 우리가 사용하는 AI의 뇌를 완성했습니다.

다음 세대로의 도약 (Transformers & LLM)

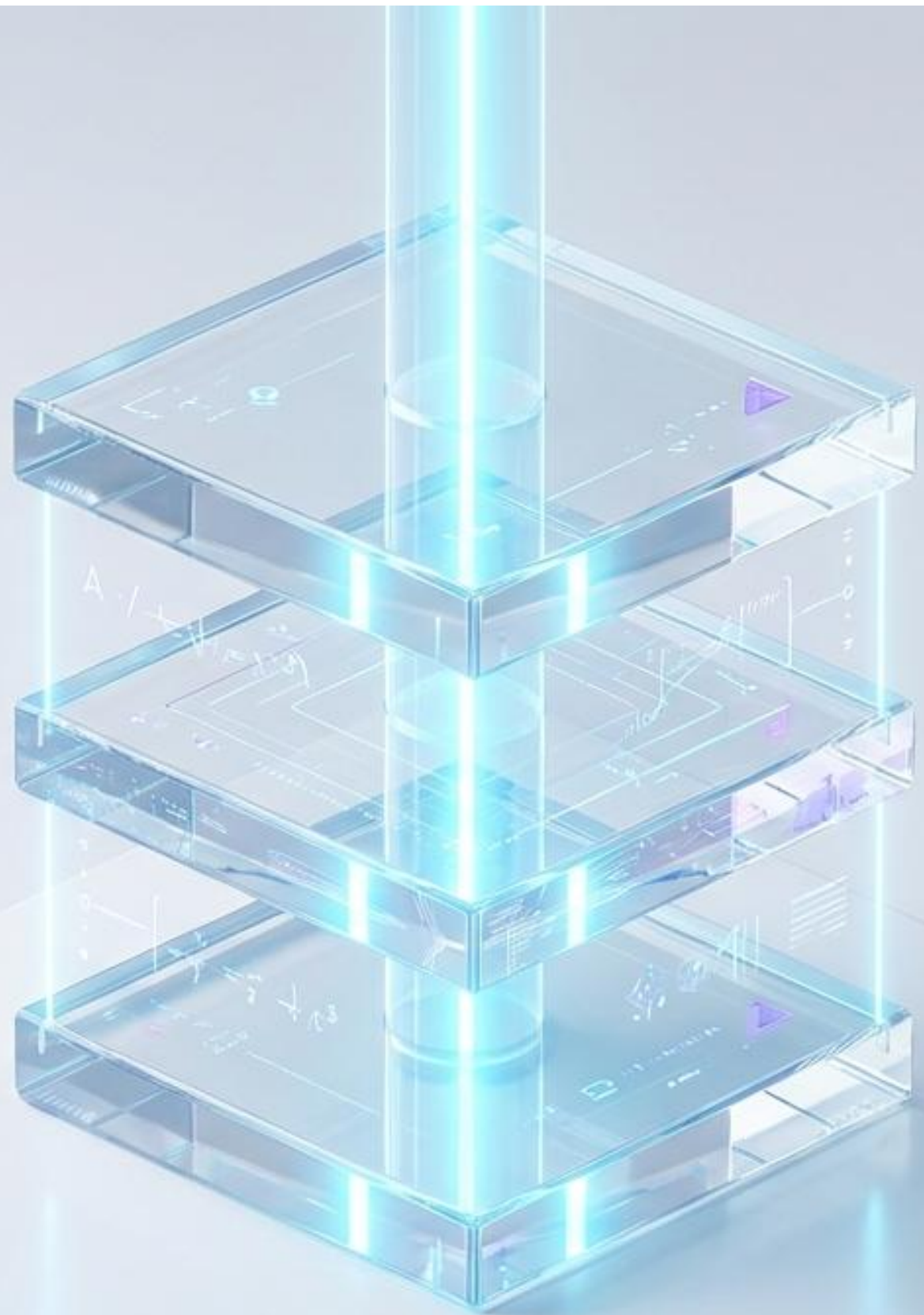
RNN의 한계:
순차 처리로 인한
장기 기억 소실

Word2Vec의 한계:
문맥의 변화를 반영하지
못하는 정적 벡터

이 모든 구조적 병목과 맥락의 부재를 한 번에 해결하기 위해,
문장 전체를 동시에 조망하는 혁명적 아키텍처 '트랜스포머(Transformer)'와
거대언어모델(LLM)의 시대가 열리게 됩니다.

투명한 AI 엔진: 트랜스포머와 셀프 어텐션의 구조

문장의 여정(The Journey of a Sentence)을 통해
추적하는 언어모델의 내부 수학과 공간적 논리



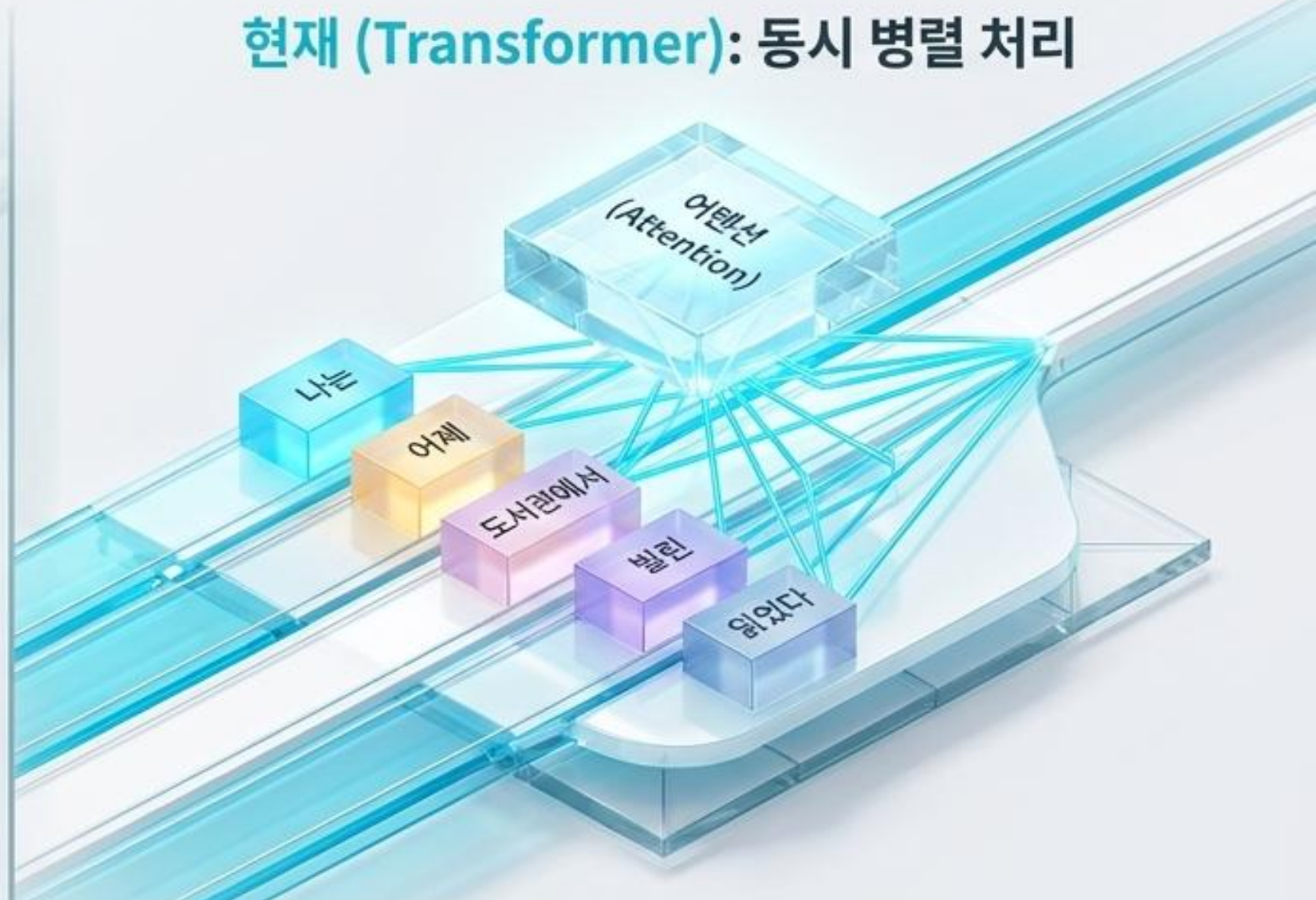
순차 처리의 한계와 병렬 처리의 혁신

과거 (RNN): 순차적 병목 현상



단어를 순서대로 처리.
문장이 길어지면 앞의 정보를 망각(장기 의존성 문제),
GPU 병렬 연산 불가.

현재 (Transformer): 동시 병렬 처리



2017년 구글 'Attention Is All You Need'.
문장 전체를 동시에 입력받아 모든 단어 간의 관계를 직접 계산.
압도적인 GPU 병렬 학습 가능.

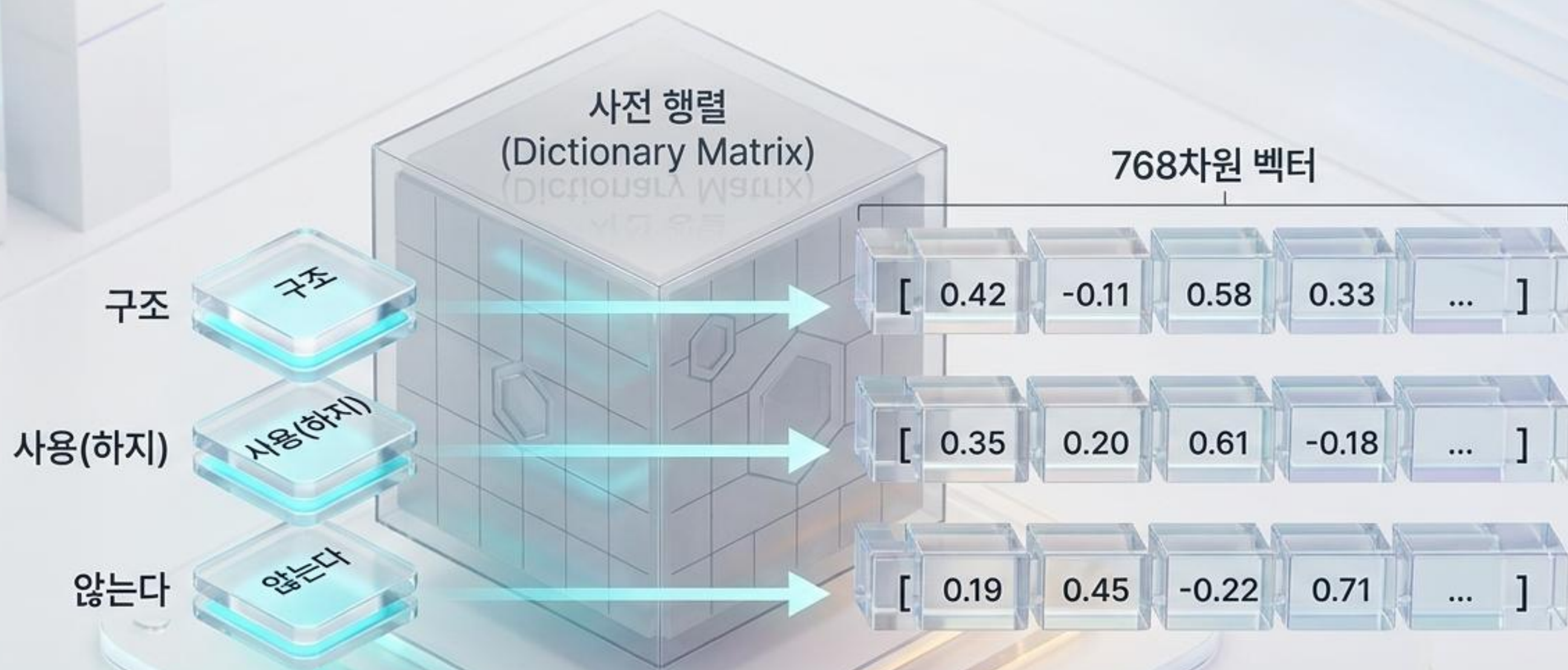
문장의 여정: 트랜스포머 블루프린트



토큰화(Tokenization): 언어의 해체와 사전(Dictionary)



토큰 임베딩: 문맥이 배제된 절대적 의미

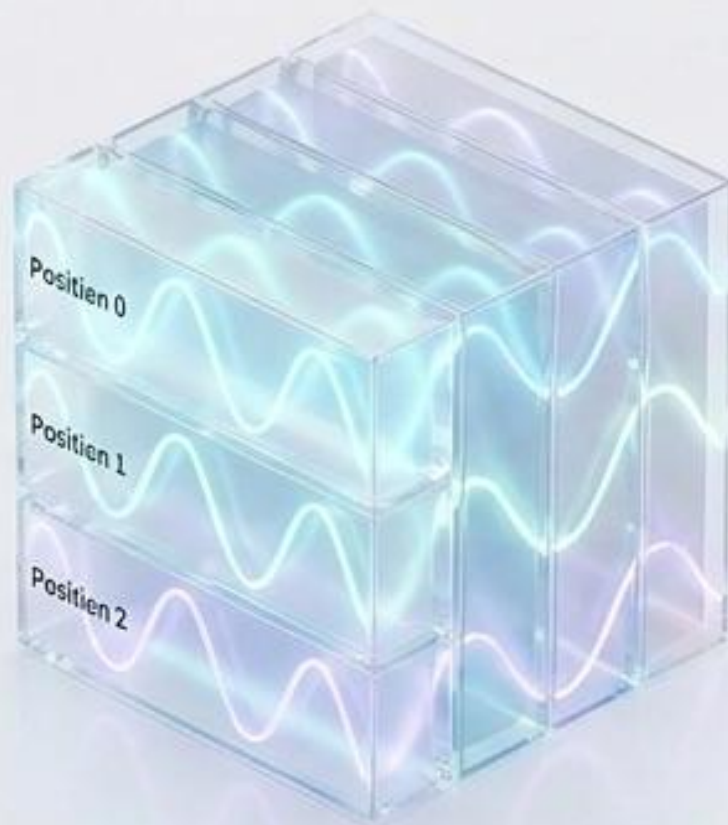


핵심 인사이트: 앞 문장의 '사용'과 뒷 문장의 '사용'은 이 단계에서 완전히 동일한 벡터를 가집니다.
아직 주변 단어(문맥)를 보지 못했기 때문입니다.

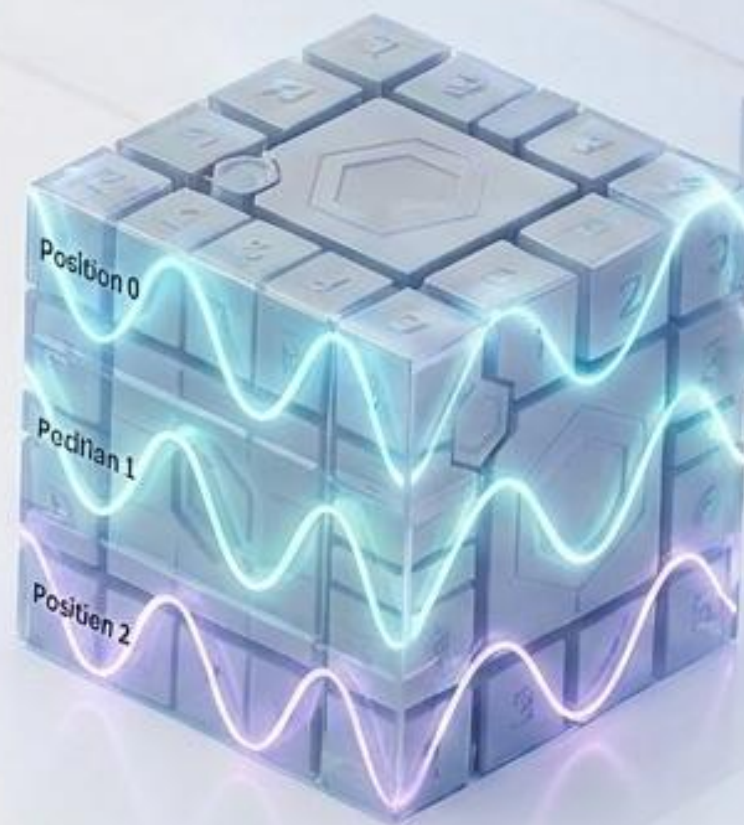
포지셔널 인코딩(Positional Encoding): 위치의 각인



정적인 토큰 임베딩 블록



위치 인코딩 벡터



위치 정보가 결합된
최종 입력 벡터 (H)

- 문제: 모든 단어를 동시에 병렬 처리하므로 순서를 자각하지 못함.
- 해결: 단어의 고유한 위치값(사인·코사인 함수)을 가진 벡터를 생성하여 기존 임베딩에 더함.
- 결과: 단어의 본질적 의미와 시간적 순서가 하나의 물리적 데이터 블록으로 결합됨.